Handoff: finishing the QPASE project report

From Iman Kalyan Chakraborty (Person A) to Person B

November 9, 2026

1 Your job

We are writing the project report together, in the style of the two sample reports in `sample_reports/`. I wrote the first half: the QPASE reproduction. You write the second half: our counter-based search index. When you finish, the report is complete. The report is `report/main.tex`, and the compiled PDF is `report/build/main.pdf`.

Section	Content	Owner	State
Abstract	summary of both halves	Person A, you revise	draft
1 Introduction	background, problem, contributions	Person A	done
2 Literature review and preliminaries	related work, notation, building blocks	Person A	done
3 The QPASE scheme	the paper's protocol, phase by phase	Person A	done
4 Implementation	modules, ring arithmetic, gaps in the paper	Person A	done
5 Evaluation of the reproduced scheme	costs, traffic, two security findings	Person A	done
6 Proposed scheme	the counter-based index	**you**	stub
7 Analysis of the proposed scheme	correctness, leakage, forward privacy, attacks	**you**	stub
8 Evaluation of the proposed scheme	notebook 07 results, search scaling	**you**	stub
9 Comparative study	both schemes; related password-based schemes	**you**	stub
10 Conclusion and future directions	summary and open work	**you**	stub
Appendix A	how every number is reproduced	Person A	done

Each of your sections already exists in `report/sections/` with its final heading, its label and a red "To do" box. Delete the box when you write the section. Sections 1 to 5 already cross-reference your labels (`sec:counter-index`, `sec:counter-analysis`, `sec:counter-evaluation`, `sec:comparison`, `sec:conclusion`), so keep those labels.

2 Set up and build

You need the project environment and a LaTeX installation with `latexmk`, `pdflatex` and `bibtex`. The report uses only standard packages, with no `biblatex`, `siunitx` or algorithm packages.

1. In the repository root, run `uv sync` if `.venv/` does not exist yet.
2. To build the report, run this in `report/`:

   ```
   latexmk main.tex
   ```

   The PDF is `report/build/main.pdf`. `report/.latexmkrc` sets PDF mode and the `build/` directory, which git ignores.

3. To check everything at once, run this in the repository root:

```
.venv/bin/python report/tools/check_report.py
```

   It checks that the generated numbers match the notebooks, builds the report, reports any error, undefined reference or undefined citation, and lists the open "To do" boxes.

Do not rerun the notebooks. The report quotes the outputs committed in `notebooks/`, which all come from one run on one machine. A rerun changes every timing, and the README and the report would then need refreshing too.

3 What to write

Target about 8 to 10 pages for Sections 6 to 10 together. The report is 20 pages now, stubs included, so it would end near 28 to 30 pages, longer than both samples (9 and 21 pages). Your primary sources are:

- `docs/counter-index-spec.md`: the full design, including the decisions the team made while building it.
- Slides 14 to 20 of `QPASE_Paper_Presentation_counter_index-2.pdf`: the motivation and the worked example.
- The code: `src/ris_sem_project/counter_index.py`, `parties/counter_index_server.py`, `parties/counter_index_user.py` and `protocol.py` (`ci_login`, `ci_outsource`, `ci_retrieve`).
- The README sections "Counter-based search index" and "Results", and notebooks 07 and 08.

3.1 Section 6: Proposed scheme

Write it like Section 3, so a reader can compare the two schemes phase by phase.

1. **Motivation.** The two problems of QPASE's search: the server keeps *dsk* and can test every later upload with it (no forward privacy), and a search tries *dsk* on every stored item.
2. **Design.** The keys $K_w = \mathrm{HKDF}(K_u, w)$, k_{ptr} and k_{ctr}, none of which leaves the device. The label of the j-th file under w, the encrypted entry that holds the item id and the file's hash, and the counter table: sealed under k_{ctr}, padded to 1024-byte blocks, versioned, signed and kept on all N servers. A small notation table helps, like `tab:notation` in Section 2.
3. **Phases.** Login step L4, the new Outsource and the new Retrieve (R1 to R5), each in a `protocol` box (snippet in Section 4.2). Register, L1 to L3 and the key update do not change, so say so in one sentence.
4. **Related designs.** The index is a counter-based encrypted dictionary in the style of Cash et al. `\cite{cash2014dynamic}`. Forward privacy as a goal comes from Stefanov et al. `\cite{stefanov2014practical}` and Bost `\cite{bost2016sophos}`.

3.2 Section 7: Analysis of the proposed scheme

1. **Correctness.** Why a search returns exactly the files under w: labels 1 to c_w, the label inside each entry, the hash check on each fetched file, and duplicates kept once.
2. **Leakage.** The table "What the server learns" from the spec or the README, with a sentence per row.
3. **Forward privacy.** The server never holds K_w, so it cannot compute the label of a later upload, and a stored search request finds only older files. Note that the scheme does not give backward privacy and that the codebase has no deletion.

4. **Attacks.** One paragraph each, as Section 4.3 of sample 1 does: a server moving an entry to another label, swapping a stored file, answering with another keyword's row (R3 drops labels it did not ask for), serving a stale or forged counter table (signature and version), and replaying a search request.
5. **Costs we accept.** From slide 20 and the spec: the counter table grows with the number of keywords, Login gains a download, Retrieve takes two round trips, two devices at once must sync the counter, and the server still sees which files a search fetches.
6. **Quantum resistance.** The index adds no new primitive. It uses the lattice PRF and HKDF, ML-DSA-44 and AES-256-GCM that QPASE already uses. Argue this informally; we have no proof.

3.3 Section 8: Evaluation of the proposed scheme

1. **Functional results from notebook 07**, in the style of Section 5.2: sizes (`\CiLabelBytes`, `\CiEntryBytes`, `\CiOutsourceRequestBytes`, `\CiSealedBytes`, `\CiUploadBytes`, `\CiSearchRequestBytes`, `\CiFetchRequestBytes`), the lookups per server (`\CiLookups`), the forward-privacy demonstration (a replayed old request finds `\CiReplayItems`, a new search finds `\CiNewSearchItems`), and a new device getting the counter table (`\CiNewDeviceVersions`).
2. **Search time against storage size from notebook 08.** The figure `search_scaling.png` and the generated table `generated/tables/search_scaling.tex` are ready (snippets in Section 4.2). Headline values: the counter index's slowest server took `\ScalingIndexMin` to `\ScalingIndexMax`, the paper's `\ScalingPaperMin` to `\ScalingPaperMax`, a speed-up of `\ScalingSpeedupMin` to `\ScalingSpeedupMax` (median `\ScalingSpeedupMedian`), with `\ScalingFailures` failures over `\ScalingChecks` checked runs.
3. **Explain the result honestly.** The speed-up does not come mainly from avoiding a slow scan: a failed AES-GCM check on a non-matching item is cheap. It comes from the paper's server running one lattice PRF chain (`\AxMedian`) per matching item, while the counter index's server only looks labels up. The scan still shows. In the generated table the paper's slowest server rises at 10^5 files at every N (for example 537 to 721 ms at $N = 3$), while the counter index stays flat. Describe that rise from the table; do not call the paper's search time constant. The user's time is about the same in both schemes.

3.4 Section 9: Comparative study

1. **QPASE against the counter index.** A table with one row per property: whether the search key leaves the device, server work per search, forward privacy, what a server learns, round trips per search, storage per file, and the user's cost per search. Use the macros above for the numbers.
2. **QPASE against related password-based schemes.** Reproduce the paper's Table V (page 14 of the paper PDF) as a table, citing the paper. My transcription is below. Check it against the paper before you use it. Every scheme in it has an entry in `references.bib`; the keys are in Section 4.3.

Scheme	Threshold	Passwords	Model	Tech.	Quantum	Retrieval	Rounds	Server / user
Bagherzandi et al. CCS'11	(t,N)	uniform	ROM	HE	no	no	2	$16/33\,T_{exe}$
Camenisch et al. CCS'12	$(2,2)$	n/a	UC	HE	no	no	1	$26/19\,T_{exe}$
Jarecki et al. ASIACRYPT'14	(t,N)	uniform	ROM	OPRF	no	no	1	$4/11\,T_{exe}$
Yi et al. ESORICS'15	(t,N)	uniform	UC	HE	no	no	1	$12/7\,T_{exe}$
Jarecki et al. EuroS&P'16	(t,N)	uniform	UC	OPRF	no	no	1	$1/4\,T_{exe}$
Jarecki et al. ACNS'17	(t,N)	uniform	UC	OPRF	no	no	1	$1/2\,T_{exe}$
Das et al. AsiaCCS	(N,N)	uniform	UC	OPRF	no	no	2	$4/10\,T_{exe}$
Chen et al. IJIS'21 (PASE)	$(2,2)$	uniform	ROM	HE	no	yes	1	$8/3\,T_{exe}$
Roy et al. ACNS'21	(t,N)	uniform	ROM	FHE	yes	no	1	$0/0$
Jiang et al. TSC'23 (QPause)	(t,N)	Zipf	ROM	FHE	yes	no	1	$0/0$
QPASE	(t,N)	Zipf	ROM	OPRF	yes	yes	1	$0/0$

T_{exe} is one exponentiation. HE is homomorphic encryption and FHE fully homomorphic encryption. The paper lists Das et al. as "ASIACCS'20" in the table, while its reference list dates the paper 2022; cite the reference list's version.

3.5 Section 10: Conclusion and future research directions

Summarise both halves in a few paragraphs: QPASE works as specified once the gaps are filled; the PRF sets its cost; PARAM II in this ring is unsafe; the linear key-update commitments reveal the shares; the counter index gives forward privacy and faster server-side search at the same user cost. Then list future work, for example: a hiding, binding commitment for the key update; parameters for PARAM II in a safe ring or in plain LWE; a faster quantum-safe PRF for labels and HKDF; multi-keyword search (Section IV-G of the paper); backward privacy and deletion; syncing the counter table across devices; constant-time sampling; and an implementation in a compiled language.

3.6 Abstract and author names

When your sections are written, revise the abstract in `sections/00-abstract.tex` so it matches them, and delete its "To do" box. In `main.tex`, replace `[Partner name]` and `[Roll number]` with yours.

4 Numbers, figures and citations

4.1 Never type a measured value

Every measured value is a LaTeX macro in `report/generated/numbers.tex`, generated from the saved notebook outputs by `report/tools/make_numbers.py`. Each line's comment names the notebook cell it came from, and Appendix A of the report lists them all. Write `\CiSealedBytes`, never `1072 B`. This keeps the report in step with the notebooks and the README, and it is how we meet the project rule that every number can be traced to a notebook cell.

If you need a value that has no macro yet, add one entry to the `NUMBERS` list in `make_numbers.py`: a macro name (letters only), the notebook number, the cell number, and a regular expression whose first group captures the value exactly as the cell prints it. For example:

```
Number("CiSearchRequestBytes", "07", 9, r"LabelSearchRequest, (\d+ B)", "label search request for
     c_w = 3"),
```

Then run `.venv/bin/python report/tools/make_numbers.py`. It stops with an error if the pattern does not match, so a wrong pattern cannot slip into the report. The paper's own numbers, such as its Table IV, are not generated: write them with a citation, as Section 5.4 does.

4.2 Snippets

A protocol phase, as in Section 3:

```
\begin{protocol}{Retrieve with the counter index}
\begin{enumerate}[R1.]
  \item If $c_w = 0$, return no files. Otherwise ...
  \item ...
\end{enumerate}
\end{protocol}
```

The search-scaling figure and table. Figures are found in `docs/figures/`:

```
\begin{figure}[ht]
\centering
\includegraphics[width=\linewidth]{search_scaling.png}
\caption{Median search time at the slowest server ... Drawn by
\texttt{notebooks/08\_search\_scaling.ipynb}, cell 7.}
\label{fig:search-scaling}
\end{figure}

\begin{table}[ht]
\centering
\small
\caption{... Source: \texttt{notebooks/08\_search\_scaling.ipynb}, cell 7.}
\label{tab:search-scaling}
\resizebox{\linewidth}{!}{\input{generated/tables/search_scaling}}
\end{table}
```

Cross-references use `\cref{...}`, which prints "Section 6" or "Table 9" for you.

4.3 Citations

`report/references.bib` holds every work the report needs. Most entries copy the QPASE paper's own reference list; a comment above each entry says so. The keys you are most likely to use are `jiang2024qpase`, `chen2021pase`, `curtmola2006sse`, `cash2014dynamic`, `stefanov2014practical`, `bost2016sophos`, `bagherzandi2011pps`, `camenisch2012pass`, `jarecki2014round`, `yi2015tpass`,

`jarecki2016highly`, `jarecki2017toppss`, `das2022dpase`, `roy2021pps` and `jiang2024qpause`. Four entries are not from the paper's list and are marked so: `lyubashevsky2010ring`, `fips204`, `dworkin2007gcm` and `bost2016sophos`. Check those four against the publishers' pages before submission. Never add a citation from memory without checking it.

5 Writing rules

Match Sections 1 to 5 so the two halves read as one report.

- Write in the first person plural, present tense, active voice: "The server looks up each label."
- One idea per sentence. Short sentences.
- No em dashes. Use a full stop or a comma instead.
- Name the real thing: the macro, the file, the notebook cell, the protocol step.
- Every claim carries its evidence: a macro, a cited table or figure, or a citation. Mark an inference as one ("we infer ..."), as Section 5.4 does.
- Use the paper's notation (Table 1 of the report) and the spec's names (K_w, k_{ptr}, k_{ctr}, c_w).

6 When you are done

The report is finished when all of these hold:

1. This command exits with status 0 (it fails while any "To do" box or name placeholder remains):

   ```
   .venv/bin/python report/tools/check_report.py --final
   ```

2. The abstract matches Sections 6 to 10.
3. You have checked the four citations that do not come from the paper's reference list.
4. You have read the whole PDF once, front to back, as a reviewer would.

Then send `report/build/main.pdf`.